Integrating GitHub Actions with Automated Unit Testing

1. Overview

In this Lab 7, you will learn how to integrate automated testing (unit and integration testing) into Github Actions. After this lab, you should be able to start incorporating automated testing into your team project.

2. Outcomes

Upon completion of this lab, you should be able to:

- Write test cases for automated unit testing using xUnit (JUnit, PHPUnit, PyUnit, etc) testing framework
- Create Github Actions to perform automated unit testing on a simple nodejs application
- Start incorporating Github Actions with automated testing into your team project
- Write test cases for automated integration and UI testing using Selenium
- Create Github Actions Workflow to perform automated integration and UI testing on a simple PHP application
- Start incorporating Github Actions Workflow into your team project

3. Server.js Code Reference

```
import express from 'express';

const app = express();

// Endpoint to return the current timestamp
app.get('/timestamp', (req, res) => {
  res.json({ timestamp: getCurrentTimestamp() });
});

// Helper function to get the current timestamp
export function getCurrentTimestamp() {
  return new Date().toISOString();
}

// Serve a simple HTML page
app.get('/', (req, res) => {
  res.send(`
    <!DOCTYPE html>
    <html lang="en">
    <head>
      <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width, initial-scale=1.0">
      <title>Browser Info</title>
    </head>
    <body>
      <h1>Browser and Timestamp Info</h1>
      <p id="browser-info">Loading browser details...</p>
      <p id="timestamp">Fetching server timestamp...</p>
      <script>
        // Display browser information
        const userAgent = navigator.userAgent;
        document.getElementById('browser-info').textContent = 'Your browser: ' + userAgent;

        // Fetch and display the server timestamp
        fetch('/timestamp')
          .then(response => response.json())
          .then(data => {
            document.getElementById('timestamp').textContent = 'Server timestamp: ' + data.timestamp;
          })
          .catch(err => {
            document.getElementById('timestamp').textContent = 'Error fetching timestamp';
          });
      </script>
    </body>
    </html>
  `);
});

// Start the server
const PORT = 3000;
const server = app.listen(PORT, '0.0.0.0', () => {
  console.log(`Server is running on port ${PORT}`);
});

// Export the server for tests
export { server };
```

4. Pre-requisite

Git

Download the provided `github-actions-nodejs-test.zip` file which contains a simple node.js app, a corresponding unit testing file and selenium integration test file and a Github Actions Workflow yml test script template (located in .github/workflows/ directory). I extracted the content to a folder name “ssd-lab7”. You can extract the zip file to your other lab directory.

Unzip and initialize it as a Git repository using the command:

```
git init
```

Next, commit with the commands:

```
git add .
```

then

```
git commit -m “Add initial files”
```

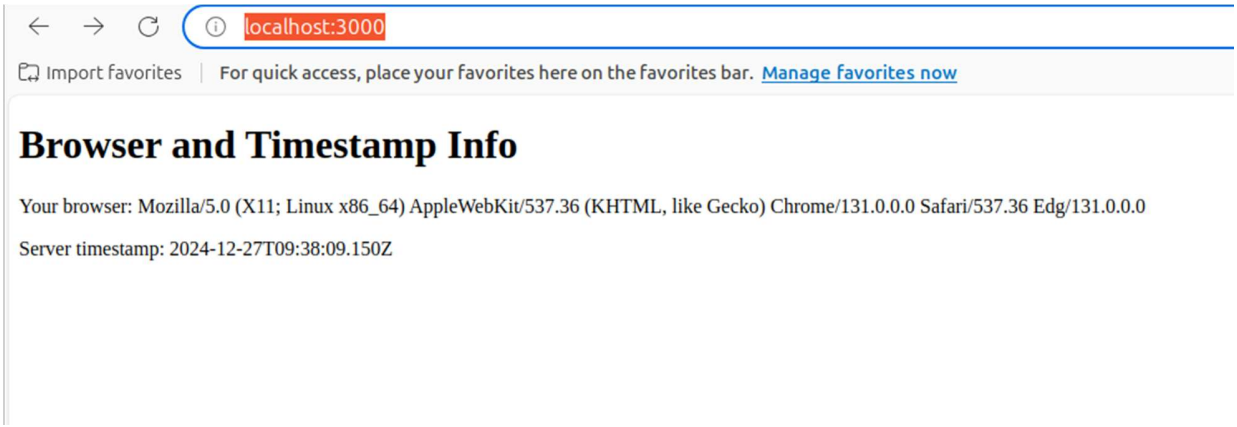
5. Run Server.js Locally

Start nodejs server

In the project root directory, run: *node src/server.js*

Open a browser, enter the url: http://localhost:3000/

You should see the following page:



6. Execute Unit Test Locally

Stop the node server.js using ctrl+c.

Review and understand the src code of tests/test.js.

You can run the unit test locally etiher using:

```
npx mocha tests/test.js
```

Or,

```
npm test
```

The definition of npm test script is found in package.json.

```
cloak@clkdev3 ~/Documents/SIT/ssd-labs-revision/ssd-lab7 (main)$npx mocha tests/test.js
Server is running on port 3000


  Timestamp Function
    ✔ should return a valid ISO timestamp
    ✔ should return the current timestamp


  2 passing (5ms)

cloak@clkdev3 ~/Documents/SIT/ssd-labs-revision/ssd-lab7 (main)$npm test

> test
> mocha tests/*.js

Server is running on port 3000


  Timestamp Function
    ✔ should return a valid ISO timestamp
    ✔ should return the current timestamp


  2 passing (3ms)
```

7. Set up Selenium Integration Test Locally

Recall that the Agile testing pyramid recommends not only automated unit testing, but also suitable amount of integration and UI testing although with lesser amount of automation.

You may be wondering how to perform automated integration and UI testing where user interactions are required. The answer lies with the use of headless browser which is a browser without GUI. Specifically, one of the popular tools available is Selenium coupled with WebDriver such as HtmlUnitDriver, FirefoxDriver or ChromeDriver, etc. In this lab, we will use Selenium together with ChromeDriver.

To help you, this Lab 7 will guide you on the use of Selenium to perform automated integration and UI testing so that you are able to start incorporating it into your team project.

Selenium automates browser testing or web app testing. In this lab, we are using selenium to test web application in a Chrome browser.

Depending on your development platform, you would need to install selenium server, selenium client web-driver and chrome driver.

For selenium server and web-driver installation, please refer to the instructions in:

- https://www.selenium.dev/
- Selenium download link: https://www.selenium.dev/downloads/
- Selenium API: https://www.selenium.dev/selenium/docs/api/java/
- HtmlUnit API: https://www.selenium.dev/htmlunit-driver/index.html?org/openqa/selenium/htmlunit/HtmlUnitDriver.html

For chrome driver, please ensure the version is for your version of chrome driver.

- https://developer.chrome.com/docs/chromedriver/get-started
- Chrome driver download link:
- https://googlechromelabs.github.io/chrome-for-testing/

Alternatively, you can start a docker container with the necessary software installed. This is the method we would use for this lab:

```
docker run -d -p 4444:4444 --name selenium-server --network host
selenium/standalone-chrome
```

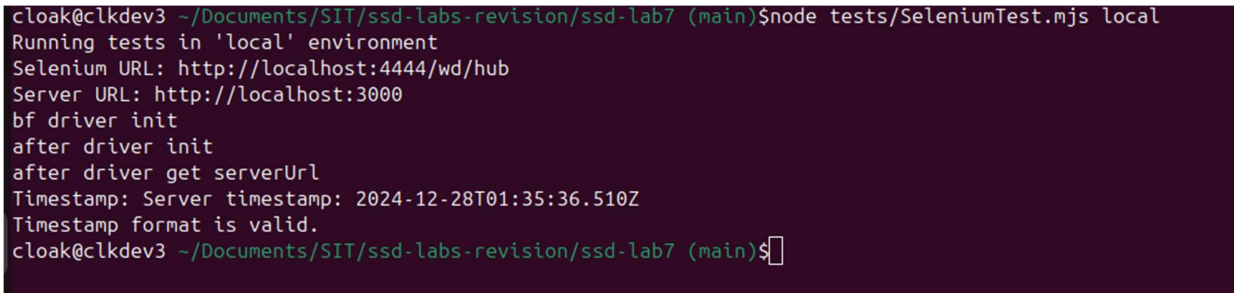
Check that the selenium server is running:

```
curl http://localhost:4444/wd/hub/status
```

8. Run Selenium Integration Test Locally

After setting up selenium and chrome driver, you can run the provided selenium test locally:

```
node tests/SeleniumTest.mjs local
```



9. Integrating Automated Unit and Integration Testing into Github Actions Workflows

Create new Github Actions Workflow

Github Action Workflow is configured mainly via yml files located under .github/workflows/ directory.

A sample configuration file named selenium-tests.yml is provided. Please review its content.

Please also take time to read through the official github action documentation.

https://docs.github.com/en/actions

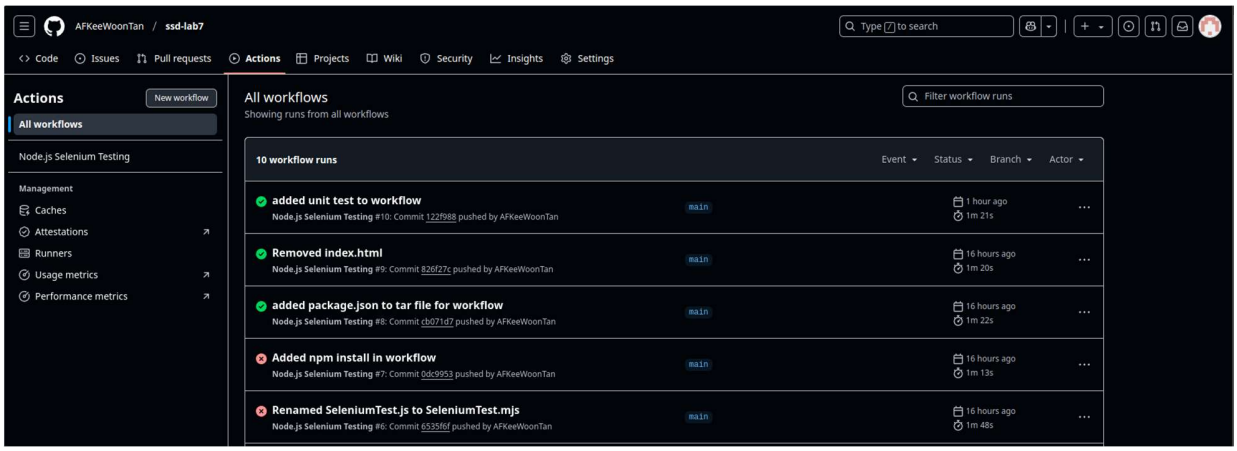
There are some important sections in the provided selenium-tests.yml file

1. The workflow is triggered based on the on: configuration in your test script. In the provided script, the workflow is triggered on pull or push requests on the main branch. You can change it to trigger only on push requests.
2. Please also note that the jobs section is divided into:
    1. a build section. This is for building the provided code. You will need to update this for your project.
    2. a test section. This is to set up the test environment. In this sample, a node image is used for the test container. And a selenium server service is started for testing. The build codes are transferred over for testing via a tar file. You will need to provide your own test configuration for your project.

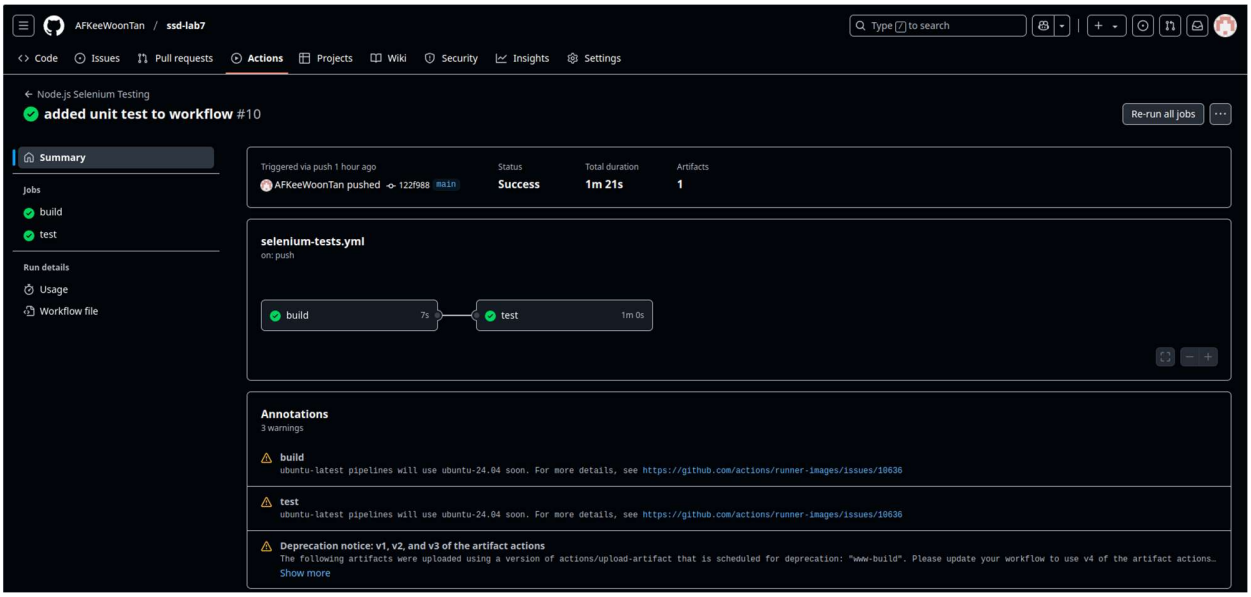
Run Testing On Github Action Workflow

Commit and push your changes to Github.

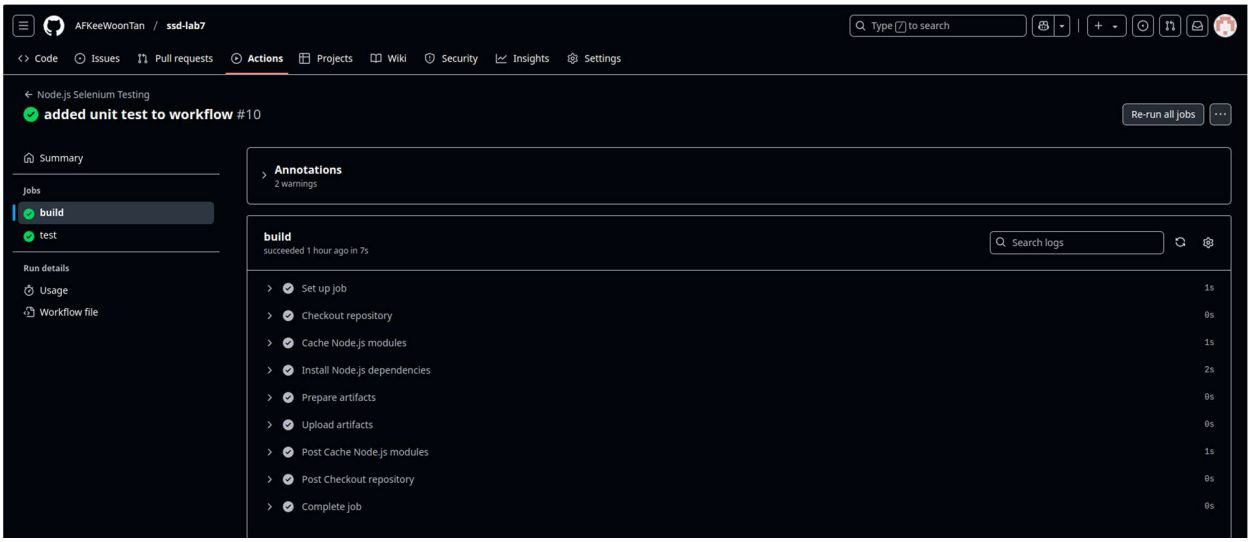
Open a browser, and navigate to your Github action page.



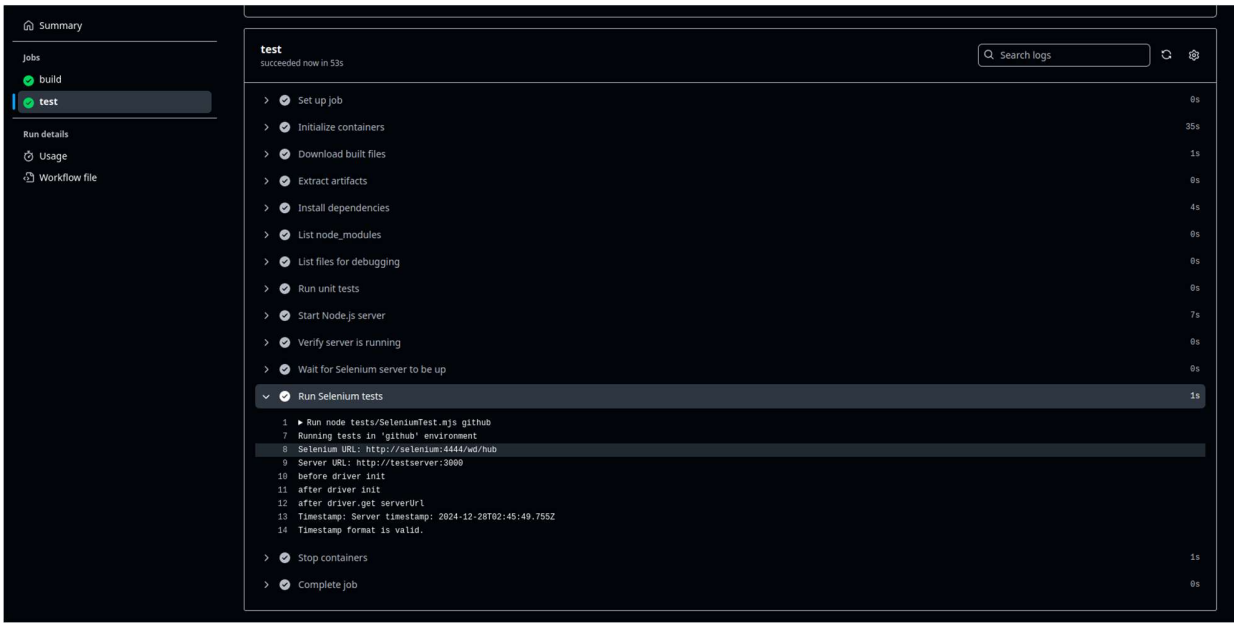
Click on the workflow run, to view its status



Click on build to view the build status



Click on test to view the test status



You can expand the section to view its details. The output depends on how you coded the test and the workflow config.

We hope that you have a better understanding of how to implement automated unit testing now. So try incorporating it into your team project. Enjoy!

END OF DOCUMENT